DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.1: Nivel Inicial — Ejercicios Prácticos

Ejercicios fundamentales de clases, atributos privados, constructores, getters y setters, la referencia this y miembros estáticos en Java.

Bienvenido al boletín de nivel inicial de la **Unidad 4: Clases y objetos**.

Estos 16 ejercicios están diseñados para asentar los conceptos fundamentales de la Programación Orientada a Objetos (POO): definición de clases, encapsulación de atributos con visibilidad `private`, métodos accesores y mutadores con validaciones de invariantes, constructores canónicos y sobrecargados con delegación `this(...)`, paso de referencias en el Heap y uso del modificador `static`.

> **Consejo de Desarrollo con IntelliJ IDEA**
> Crea un proyecto en IntelliJ IDEA y coloca cada clase en su propio archivo `.java`. Practica la generación automática de constructores, getters y setters mediante el atajo `Alt + Insert` (o clic derecho → *Generate*...).

Ejercicio 1: La Clase `Libro`, Delegación con `this(...)` y Encapsulamiento Básico

Enunciado:
Diseña una clase llamada `Libro` que contenga los atributos privados: `titulo` (`String`), `autor` (`String`) y `numeroPaginas` (`int`).

- Define un constructor canónico que inicialice los tres atributos, validando que el título y el autor no sean nulos ni cadenas vacías, y que el número de páginas sea mayor o igual a 0 (lanzando `IllegalArgumentException` si no se cumplen).
- Añade un **constructor sin parámetros** que delegue en el constructor principal utilizando `this("Sin título", "Desconocido", 0)`.
- Implementa todos los métodos getters y setters con las mismas validaciones.
- Añade un método `public boolean esLargo()` que devuelva `true` si el libro tiene más de 300 páginas y `false` en caso contrario.
- En una clase `Main`, crea tres objetos `Libro`: `"Ficciones"` de `"Borges"` (200 páginas), `"1984"` de `"Orwell"` (328 páginas) y un tercer libro usando el constructor sin parámetros. Muestra por consola sus datos usando los getters, comprueba cuáles son largos con `esLargo()`, modifica el número de páginas de uno de ellos con su setter y vuelve a mostrarlo.

Ejercicio 2: Geometría y Contador Estático de Instancias con `Rectangulo`

Enunciado:
Crea una clase `Rectangulo` con dos atributos de instancia privados: `base` y `altura` (ambos de tipo `double`), y un atributo de clase privado `private static int numeroDeRectangulos = 0;`.

- El constructor debe rechazar valores menores o iguales a cero e incrementar `numeroDeRectangulos` cada vez que se instancie un rectángulo válido.

- Implementa los métodos `public double calcularArea()` (`base × altura`) y `public double calcularPerimetro()` (`2 × (base + altura)`).
- Implementa un método `public boolean esCuadrado()` que determine si la base y la altura son idénticas.
- Añade un método `public void escalar(double factor)` que multiplique ambas dimensiones por un factor positivo.
- Añade un método estático `public static int getNumeroDeRectangulos()` que devuelva cuántos rectángulos se han creado en total. Demuestra su funcionamiento en `main` creando 3 rectángulos y mostrando el contador global.

Ejercicio 3: La Clase `Circulo` y Constantes Matemáticas

Enunciado:
Diseña una clase `Circulo` con un atributo privado `radio` (double).

- El constructor debe asegurar que el radio sea positivo.
- Implementa `calcularArea()` y `calcularPerimetro()` haciendo uso de la constante estática oficial `Math.PI`.
- Proporciona un constructor por defecto sin parámetros que asigne un radio de `1.0` delegando en el constructor principal mediante `this(1.0)`.

Ejercicio 4: Conversor Térmico con la Clase `Termometro`

Enunciado:
Crea una clase `Termometro` que almacene internamente la temperatura en grados Celsius (`temperaturaCelsius` de tipo `double`).

- El constructor debe verificar que la temperatura no sea inferior al cero absoluto (-273.15 °C).
- Añade métodos para consultar la temperatura en escala Fahrenheit: `public double getFahrenheit()` y en escala Kelvin: `public double getKelvin()`.
- Añade métodos `incrementar(double delta)` y `decrementar(double delta)`.

Ejercicio 5: Coordenadas Cartesianas con la Clase `Punto2D`

Enunciado:
Modela un punto en el plano bidimensional con la clase `Punto2D`:

- Atributos privados: `x` e `y` (double).
- Constructor por defecto que sitúe el punto en el origen `(0.0, 0.0)` mediante `this(0.0, 0.0)`.
- Constructor parametrizado `public Punto2D(double x, double y)`.
- Método `public void desplazar(double dx, double dy)` que traslade las coordenadas.
- Método `public double distanciaAlOrigen()` que devuelva la distancia euclídea respecto al `(0,0)` (`√(x² + y²)`).

Ejercicio 6: La Máquina Cafetera

Enunciado:
Diseña una clase `Cafetera` para gestionar el servicio de café en una oficina:

- Atributos privados: `capacidadMaxima` (c.c. de café, por defecto 1000) y `cantidadActual` (c.c. de café que contiene).
- Constructor predeterminado: capacidad máxima 1000 c.c. y cafetera vacía (0 c.c.).
- Constructor con capacidad máxima dada (inicialmente vacía).
- Constructor con capacidad máxima y cantidad actual (si la cantidad actual supera la máxima, se ajusta al máximo).
- Métodos:
  - `llenarCafetera()`: llena la cafetera a su máxima capacidad.
  - `servirTaza(int capacidadTaza)`: descuenta el café servido. Si no hay suficiente para llenar la taza, sirve lo que quede y deja la cafetera a 0. Devuelve la cantidad real servida.
  - `vaciarCafetera()`: pone la cantidad actual a 0.
  - `agregarCafe(int cantidad)`: añade café sin sobrepasar la capacidad máxima.

Ejercicio 7: Cronómetro Simple con la Clase `Reloj`

Enunciado:
Modela la clase `Reloj` con tres atributos enteros privados: `hora` (0-23), `minuto` (0-59) y `segundo` (0-59).

- Constructor canónico con validación rigurosa de rangos.
- Constructor sin argumentos que inicialice el reloj a las `00:00:00`.
- Método `public void ticTac()` que adelante el reloj exactamente un segundo, gestionando adecuadamente los cambios de minuto, hora y el paso de medianoche (`23:59:59` → `00:00:00`).
- Método `public String getHoraFormateada()` que devuelva la hora en formato `HH:mm:ss` con dos dígitos por campo (p. ej. `"09:05:03"`).

Ejercicio 8: Identificadores Autoincrementales y Sueldo Protegido con `Empleado`

Enunciado:
Diseña una clase `Empleado` con:

- Un atributo estático privado `contadorSecuencial` inicializado en 1000 y un `private static String passwordMaestra = "RRHH2025";` común a toda la empresa.
- Un atributo de instancia inmutable `final int idEmpleado`.
- Atributos de instancia privados: `nombre` (String obligatorio), `edad` (int, debe ser siempre `>= 18`), `departamento` (String) y `sueldo` (double).
- Cada vez que se cree un nuevo empleado, el constructor debe asignarle automáticamente `++contadorSecuencial` y validar que el nombre no esté vacío y la edad sea `>= 18`.
- El sueldo solo podrá consultarse o modificarse mediante los métodos `getSueldo(String password)` y `setSueldo(double nuevoSueldo, String password)`, que comprueban primero si la contraseña coincide con `passwordMaestra` (lanzando `SecurityException` en caso contrario).

Ejercicio 9: La Clase `Contador` y Encadenamiento con `this`

Enunciado:
Implementa una clase `Contador` con un atributo entero privado `valor`.

- Constructor sin parámetros: inicializa en 0.

- Constructor con valor inicial.
- Constructor de copia: `public Contador(Contador otro)` que replique el valor de otra instancia.
- Métodos `incrementar()`, `decrementar()` e `incrementar(int cantidad)`.
- Implementa los métodos devolviendo la referencia `this` para posibilitar llamadas fluidas encadenadas: `contador.incrementar().incrementar(5).decrementar();`

Ejercicio 10: Array de Objetos Alumnos

Enunciado:
Utilizando la clase `Alumno` (con `nombre` y `nota` entre 0.0 y 10.0):

- En el método `main`, declara e inicializa un array de 5 alumnos con nombres y calificaciones distintas.
- Diseña un método estático `public static double calcularNotaMedia(Alumno[] lista)` que calcule la media del grupo.
- Diseña un método estático `public static Alumno obtenerMejorAlumno(Alumno[] lista)` que localice y devuelva la referencia al alumno con la calificación más alta.

Ejercicio 11: Control de Vehículo con la Clase `Coche`

Enunciado:
Modela una clase `Coche` con los atributos: `marca`, `modelo`, `motorEncendido` (boolean) y `velocidadActual` (int, en km/h).

- El constructor recibe marca y modelo; el coche arranca apagado y con velocidad 0.
- Métodos:
  - `arrancar()`: enciende el motor. No se puede arrancar si ya está encendido.
  - `apagar()`: apaga el motor, solo si la velocidad es 0 (no se puede apagar en movimiento).
  - `acelerar(int incremento)`: incrementa la velocidad si el motor está encendido, hasta un máximo de 200 km/h.
  - `frenar(int decremento)`: reduce la velocidad sin bajar de 0 km/h.

Ejercicio 12: Generador y Validador de Contraseñas

Enunciado:
Crea la clase `Password`:

- Atributos: `longitud` (por defecto 8) y `valor` (String con la contraseña).
- Constructor por defecto: genera una contraseña aleatoria de 8 caracteres.
- Constructor con longitud personalizada: genera una contraseña con la longitud indicada.
- Método `public boolean esFuerte()`: devuelve `true` si la contraseña contiene al menos 2 mayúsculas, 1 minúscula y 2 dígitos numéricos.
- Método `public void regenerar()`: genera un nuevo valor aleatorio manteniendo la longitud actual.

Ejercicio 13: Visibilidad `private` vs. `public` y Auditoría de Código con Errores

Enunciado:
Responde de forma razonada y corrige los siguientes casos prácticos:

1. **Cuestión conceptual:** ¿Qué diferencia existe entre declarar un atributo como `private` y declararlo como `public`? ¿Por qué en Programación Orientada a Objetos se recomienda declarar siempre los atributos como `private` y exponer getters/setters?
2. **Auditoría 1 (Contexto estático inválido):** Un compañero propone añadir a la clase `Empleado` el método `public static String getNombre() { return nombre; }` argumentando que «hace lo mismo para todos los objetos». Explica por qué no compila y por qué es conceptualmente erróneo.
3. **Auditoría 2 (Caza de errores en `Punto2D`):** Analiza el siguiente código defectuoso, enumera todos los fallos léxicos, sintácticos y de diseño que contiene (fíjate en el falso constructor `void`, el sombreamiento `x = x` sin `this`, la visibilidad y la impresión) y reescríbelo corregido:

```
Public class empleado {
    public double x, y;
    public void Punto2D(double x, double y) {
        x = x;
        y = y;
    }
}
```

Ejercicio 14: Traza de Referencias Compartidas en el Heap

Enunciado:
Sin ejecutar el programa, deduce qué mostrará exactamente por pantalla el siguiente código y explica paso a paso qué ocurre en la memoria **Stack** y en la memoria **Heap**:

```
Libro l1 = new Libro("Ficciones", "Borges", 200);
Libro l2 = l1;
l2.setTitulo("El Aleph");
System.out.println(l1.getTitulo());
System.out.println(l1 == l2);
```

A continuación, explica cómo tendríamos que crear `l2` si quisiéramos tener un segundo objeto `Libro` completamente independiente en memoria con los mismos datos iniciales que `l1`.

Ejercicio 15: La Clase `FormateadorBanner`: Encapsulamiento y Sobrecarga Orientada a Objetos

Enunciado:
Crea una clase `FormateadorBanner` para generar carteles y textos enmarcados en consola aplicando sobrecarga de métodos de instancia y variables de clase:

- Atributos de instancia privados: `caracterBorde` (`char`, por defecto `'*'`) y `anchoMinimo` (`int`, debe ser como mínimo 10).
- Atributo de clase privado: `private static int totalBannersGenerados = 0;`.
- Constructores:
  - Constructor por defecto que asigne `'*'` y ancho mínimo 30 delegando en el constructor parametrizado mediante `this('*', 30)`.
  - Constructor parametrizado `public FormateadorBanner(char caracterBorde, int anchoMinimo)`.
- Métodos sobrecargados de instancia:

  - `public void imprimirBanner(String texto)`: imprime el texto centrado entre dos líneas horizontales formadas por el `caracterBorde`, asegurando el ancho mínimo e incrementando el contador global `totalBannersGenerados`.
  - `public void imprimirBanner(String texto, char caracterEspecial)`: sobrecarga que utiliza temporalmente un carácter especial para ese cartel concreto, reutilizando la lógica interna.
  - `public void imprimirLineas(String... lineas)`: acepta un número variable de líneas (*varargs*) y las imprime todas dentro de un único recuadro coordinado.
- Método estático `public static int getTotalBannersGenerados()` para consultar el total global de carteles creados por cualquier instancia.
- En `main`, crea dos instancias (una con `'='` y otra por defecto), genera varios carteles y banners multilínea, y comprueba el contador estático.

Ejercicio 16: Modelado de Objetos con Estado: Las Clases `Dado` y `Cubilete`

Enunciado:
En lugar de funciones procedimentales sueltas, modela un simulador de juegos de azar aplicando Programación Orientada a Objetos:

1. Diseña la clase `Dado`:
   - Atributos privados: `numeroCaras` (entero, por defecto 6, validando que sea `>= 2`), `ultimoValor` (entero) y una instancia de `java.util.Random`.
   - Constructor por defecto (6 caras) y constructor parametrizado `public Dado(int numeroCaras)`.
   - Método `public int lanzar()`: genera un número aleatorio entre `1` y `numeroCaras`, actualiza `ultimoValor` y lo devuelve.
   - Getter `public int getUltimoValor()` y método `toString()` que devuelva `"Dado de [N] caras (última tirada: [V])"`.
2. Diseña la clase `Cubilete`:
   - Atributo privado: un array de dados `private Dado[] dados`.
   - Constructor `public Cubilete(int cantidadDados, int carasPorDado)`: inicializa el array con la cantidad indicada de objetos `Dado`.
   - Método `public int tirarTodos()`: lanza todos los dados del cubilete y devuelve la suma total obtenida.
   - Método `public int tirarHastaSuma(int sumaObjetivo)`: lanza repetidamente el cubilete hasta que la suma coincida con `sumaObjetivo` (validando que el objetivo sea matemáticamente posible), devolviendo cuántas tiradas han sido necesarias.
3. En `main`, crea un cubilete con 2 dados de 6 caras, calcula cuántas tiradas se necesitan para sacar un 12, muestra el estado de cada dado mediante sus métodos y prueba también un dado de 20 caras (`D20`).

> **¿Necesitas consultar las soluciones?**
> Revisa el código completo comentado en el Boletín 4.1 Resuelto.